

HW2(FEA): Appiah Francis

- (iii) The code is Implemented with a uniform mesh $h_j = h = \frac{1}{N+1}$. From (i) the exact solution is given by

$$u(x) = \frac{e^{1/\varepsilon}}{e^{1/\varepsilon} - 1} - \frac{e^{1/\varepsilon}}{e^{1/\varepsilon} - 1} e^{x/\varepsilon}$$

with it's derivative

$$u'(x) = -\frac{1}{\varepsilon} \frac{e^{1/\varepsilon}}{e^{1/\varepsilon} - 1} e^{x/\varepsilon}.$$

Below is the plot of the exact solution againts the approximate solution $U_I(x)$.

Table 1: Errors and convergence orders when $\varepsilon = 0.5$

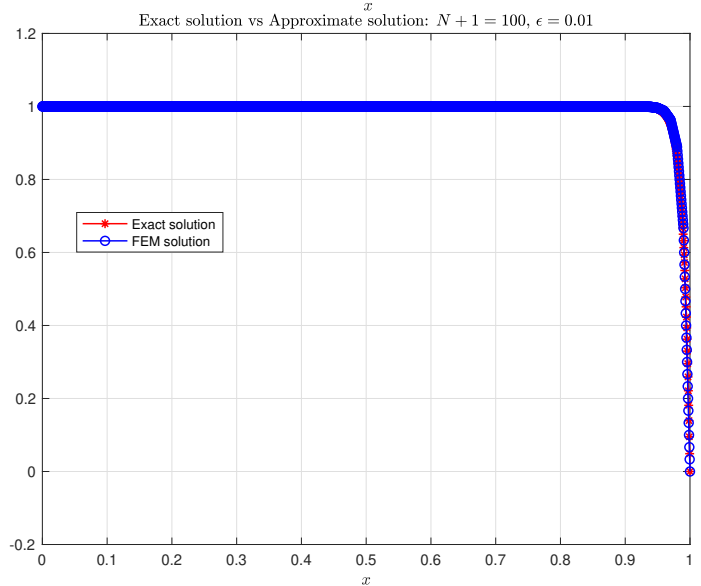
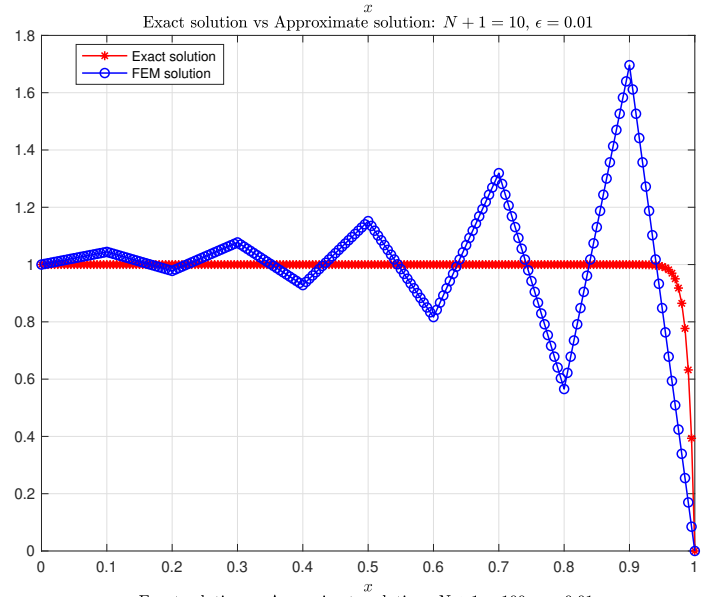
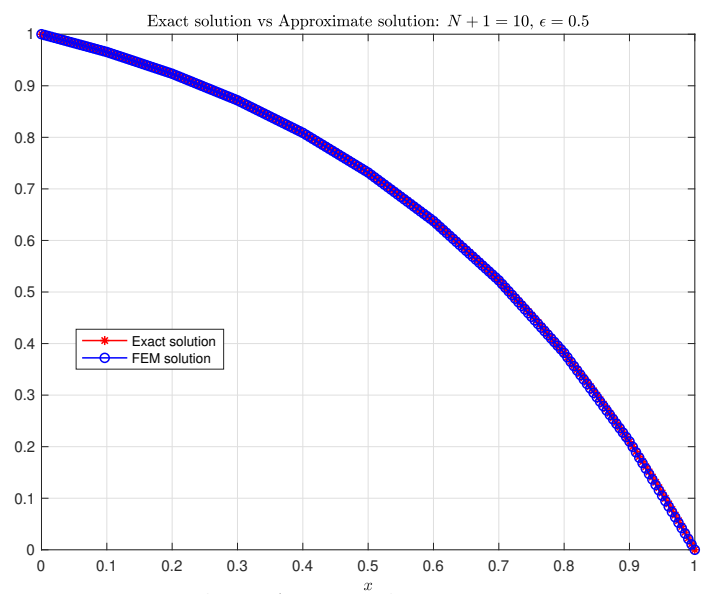
$N + 1$	$\ e_h(\cdot)\ _{L^2([0,1])}$	Order	$\ e_h(\cdot)\ _{H^1([0,1])}$	Order
10	1.77E-03	-	7.46E-02	-
100	1.77E-05	2.00	7.33E-03	1.01
1000	1.77E-07	2.00	7.32E-04	1.00
10000	1.77E-09	2.00	7.31E-05	1.00

Table 2: Errors and convergence orders when $\varepsilon = 0.01$

$N + 1$	$\ e_h(\cdot)\ _{L^2([0,1])}$	Order	$\ e_h(\cdot)\ _{H^1([0,1])}$	Order
10	2.01E-01	-	1.64E+01	-
100	5.10E-03	1.59	2.35E+00	0.84
1000	5.26E-05	1.99	2.27E-01	1.02
10000	5.26E-07	2.00	2.26E-02	1.00

Table 3: Errors and convergence orders when $\varepsilon = 0.001$

$N + 1$	$\ e_h(\cdot)\ _{L^2([0,1])}$	Order	$\ e_h(\cdot)\ _{H^1([0,1])}$	Order
10	2.59E+00	-	1.79E+02	-
100	6.19E-02	1.62	5.18E+01	0.54
1000	1.61E-03	1.58	7.45E+00	0.84
10000	1.66E-05	1.99	7.18E-01	1.02



1 Conclusion

When $\varepsilon = 0.5$ the diffusion part dominates, and the solution is smooth across the domain. Using a uniform mesh works well for all values of $N + 1$. We can see from the table the L^2 norm and H^1 norm convergence is 2nd and 1st order respectively. This is consistent with the theoretical error estimates for piecewise linear finite element methods. When ε is very small, the equation behaves approximately like $u'(x) = 0$, which gives a constant solution. For most of the domain $x < 1$, the solution satisfies

$$u(x) \approx 1.$$

However, near $x = 1$, the solution rapidly decreases to satisfy the boundary condition $u(1) = 0$. A uniform mesh may not resolve this layer unless the mesh size satisfies $h \ll \varepsilon$. The convergence rates deteriorate initially because the uniform mesh cannot adequately resolve this issue. The error is largest near $x = 1$, where the solution has steep gradients. This steep gradients creates very large derivatives. This is evident from table 2 and 3 as the H^1 norm errors are huge when the mesh is not refined enough. For $\varepsilon = 0.01$ and $N + 1 = 10$ we see FEM struggles to approximate the exact solution, but as the mesh is refined, the error decreases and the expected convergence rates are eventually recovered. The convection–diffusion problem demonstrates the challenges of solving singularly perturbed equations numerically. When ε is small, the solution develops a boundary layer (perturbation methods), which requires fine mesh resolution for accurate approximation.

Listing 1: main.m

```

1  clc;close all;
2  par.a = 0; par.b = 1; par.eps = 0.5;
3  Nvalue = [9,99,999,9999];
4  par.ccoeffOpt = 1;
5  par.testOption = 1;
6  par.debug = 0;
7  %%
8  l2norm = zeros(length(Nvalue),1);
9  linfnorm = zeros(length(Nvalue),1);
10 enorm = zeros(length(Nvalue),1);
11
12 xq = cell(length(Nvalue),1);
13 error = cell(length(Nvalue),1);
14 Uexact = cell(length(Nvalue),1);
15 UI = cell(length(Nvalue),1);
16 for j = 1:length(Nvalue)
17     par.N = Nvalue(j);
18     gpar = twoPointBVP(par);
19     xq{j} = gpar.xq;
20     error{j} = gpar.eh_vec;
21     Uexact{j} = gpar.u_vec;
22     UI{j} = gpar.uh_vec;
23     l2norm(j) = gpar.L2Norm;
24     linfnorm(j) = gpar.LinfNorm;
25     enorm(j) = gpar.ENorm;
26 end
27 %% make plots and convergence study
28 if par.eps == 0.5
29     figure;

```

```

30 plot(xq{1}, Uexact{1}, 'r-*', 'LineWidth', 1)
31 hold on
32 plot(xq{1}, UI{1}, 'b-o', 'LineWidth', 1)
33 hold off
34 grid on;
35 xlim([0 1]);
36 legend('Exact_solution', 'FEM_solution', 'Location','best')
37 xlabel('$x$', 'Interpreter', 'latex');
38 title(sprintf('Exact_solution_vs_Approximate_solution: %d$, %d\epsilon=%g$', Nvalue(1)+1, par.
    eps), ...
39     'Interpreter', 'latex')
40 filename = sprintf('img/solplot_N%d_eps%g.eps', Nvalue(1)+1, par.eps);
41 print('-depsc2', filename);
42 end
43
44 if par.eps == 0.01
45 figure;
46 plot(xq{1}, Uexact{1}, 'r-*', 'LineWidth', 1)
47 hold on
48 plot(xq{1}, UI{1}, 'b-o', 'LineWidth', 1)
49 hold off
50 grid on;
51 xlim([0 1]);
52 legend('Exact_solution', 'FEM_solution', 'Location','best')
53 xlabel('$x$', 'Interpreter', 'latex');
54 ylabel('$|e_h(x)|$', 'Interpreter', 'latex');
55 title(sprintf('Exact_solution_vs_Approximate_solution: %d$, %d\epsilon=%g$', Nvalue(1)+1, par.
    eps), ...
56     'Interpreter', 'latex')
57 filename = sprintf('img/solplot_N%d_eps%g.eps', Nvalue(1)+1, par.eps);
58 print('-depsc2', filename);
59
60 figure;
61 plot(xq{2}, Uexact{2}, 'r-*', 'LineWidth', 1)
62 hold on
63 plot(xq{2}, UI{2}, 'b-o', 'LineWidth', 1)
64 hold off
65 grid on;
66 xlim([0 1]);
67 legend('Exact_solution', 'FEM_solution', 'Location','best')
68 xlabel('$x$', 'Interpreter', 'latex');
69 title(sprintf('Exact_solution_vs_Approximate_solution: %d$, %d\epsilon=%g$', Nvalue(2)+1, par.
    eps), ...
70     'Interpreter', 'latex')
71 filename = sprintf('img/solplot_N%d_eps%g.eps', Nvalue(2)+1, par.eps);
72 print('-depsc2', filename);
73 end
74 %% compute convergence
75 l2conv = zeros(length(Nvalue)-1,1);
76 linfconv = zeros(length(Nvalue)-1,1);
77 Econv = zeros(length(Nvalue)-1,1);
78 for j = 1 : length(Nvalue)-1
79     l2conv(j) = log(l2norm(j)/l2norm(j+1))/log(10);
80     linfconv(j) = log(linfnorm(j)/linfnorm(j+1))/log(10);
81     Econv(j) = log(enorm(j)/enorm(j+1))/log(10);
82 end

```

Listing 2: twoPointBVP.m

```

1 function [gpar] = twoPointBVP(par)
2 a = par.a; b = par.b; eps = par.eps;
3 N = par.N; debug = par.debug;
4 h = 1/(N + 1);
5 x = zeros(N+2,1);
6 c = zeros(N+2,1);
7 x(1) = a;
8 for j = 1 : N+1
9     x(j+1) = h + x(j);
10 end
11 UTRUE = @(x) (1-(exp((x-1)/eps)))/(1-exp(-1/eps));
12 UTRUEX = @(x) - 1/eps * (exp((x-1)/eps))/(1-exp(-1/eps));
13 alpha = 1;
14 beta = 0;
15 A = formMatrix(x,h,par);
16 %form vector F
17 F = zeros(N,1);
18 F(1) = eps*alpha/h + alpha/2;
19 F(N) = eps*beta/h - beta/2;
20 % solve for c
21 c(1) = alpha;
22 c(N+2) = beta;
23 c(2:N+1) = A\F;
24
25 %% calculate errors norms
26 M = 3;
27 L2Norm = 0;
28 LinfNorm = 0;
29 ENorm = 0;
30 for j = 1: N+1
31     for m = 1:M
32         idx = x(j) + h*m/M;
33         e_h = UhFunc(idx,j,x,h,c) - UTRUE(idx);
34         e_hx = UhFuncx(j,h,c) - UTRUEX(idx);
35         L2Norm = L2Norm + e_h^2* h/M;
36         LinfNorm = max(LinfNorm ,abs(e_h));
37         ENorm = ENorm + e_hx^2* h/M;
38     end
39 end
40 gpar.L2Norm = sqrt(L2Norm);
41 gpar.ENorm = sqrt(ENorm);
42 gpar.LinfNorm = LinfNorm;
43 %% FEM solution function
44 Mq = 20;
45 xq_vec = x(1);
46 % store FEM and exact solutions
47 uh_vec = UhFunc(x(1),1,x,h,c);
48 u_vec = UTRUE(x(1));
49 eh_vec = u_vec -uh_vec;
50 for j = 1:N+1
51     for m = 1:Mq
52         idxq = x(j) + h*m/Mq;
53         % compute solutions
54         uh = UhFunc(idxq,j,x,h,c);
55         u = UTRUE(idxq);
56         % error
57         e_h = uh - u;
58         % store values

```

```

59     xq_vec = [xq_vec; idxq];
60     eh_vec = [eh_vec; e_h];
61     uh_vec = [uh_vec; uh];
62     u_vec = [u_vec; u];
63     end
64 end
65 gpar.xq = xq_vec;
66 gpar.eh_vec = eh_vec;
67 gpar.u_vec = u_vec;
68 gpar.uh_vec = uh_vec;
69 %%
70 if debug == 1
71     % for testing uh function at the nodes
72     uh = zeros(length(x),1);
73     uexact = zeros(length(x),1);
74     for j = 1:N+1
75         uh(j) = UhFunc(x(j),j,x,h,c);
76         uexact(j) = UTRUE(x(j));
77     end
78     uh(N+2) = UhFunc(x(N+2),N+1,x,h,c);
79     uexact(N+2) = UTRUE(x(N+2));
80     figure(1)
81     plot(x, uh, 'b-*', 'LineWidth', 1.5);
82     hold on
83     plot(x, uexact, 'r-', 'LineWidth', 1.5);
84     hold on
85     plot(x, c, 'k--', 'LineWidth', 1.5);
86     hold off;
87     grid on;
88 end
89 return
90 end

```

Listing 3: formMatrix.m

```

1 function A = formMatrix(x,h,par)
2 % Define Matrix A
3 coeffOpt = par.coeffOpt;
4 N = par.N;
5 A = zeros(N,N);
6 if coeffOpt == 1
7     % First row
8     A(1,1) = par.eps*(1/h + 1/h);
9     A(1,2) = -par.eps/h + 1/2;
10
11     % Interior rows
12     for i = 2:N-1
13         A(i,i-1) = -par.eps/h -1/2;
14         A(i,i) = par.eps*(1/h + 1/h);
15         A(i,i+1) = -par.eps/h + 1/2;
16     end
17
18     % Last row
19     A(N,N-1) = -par.eps/h -1/2;
20     A(N,N) = par.eps*(1/h + 1/h);
21 else
22     % First row
23     A(1,1) = (1+0.5*(x(1)+x(2)))/h(1) + (1+0.5*(x(2)+x(3)))/h(2);
24     A(1,2) = -(1+0.5*(x(2)+x(3)))/h(2);

```

```

25
26 % Interior rows
27 for i = 2:N-1
28     A(i,i-1) = -(1+0.5*(x(i)+x(i+1)))/h(i);
29     A(i,i) = (1+0.5*(x(i)+x(i+1)))/h(i) + (1+0.5*(x(i+1)+x(i+2)))/h(i+1);
30     A(i,i+1) = -(1+0.5*(x(i+1)+x(i+2)))/h(i+1);
31 end
32
33 % Last row
34 A(N,N-1) = -(1+0.5*(x(N)+x(N+1)))/h(N);
35 A(N,N) = (1+0.5*(x(N)+x(N+1)))/h(N) + (1+0.5*(x(N+1)+x(N+2)))/h(N+1);
36 end
37 A = sparse(A);
38 return
39 end

```

Listing 4: $U_h(x)$ function

```

1 function U_h = UhFunc(a,j,x,h,c)
2 U_h = (c(j)*(x(j+1)-a)+c(j+1)*(a-x(j)))/h;
3 return
4 end

```

Listing 5: $U'_h(x)$ function

```

1 function Uh_x = UhFuncx(j,h,c)
2 Uh_x = (-c(j)+c(j+1))/h;
3 return
4 end

```